UNIVERSITAT POLITÈCNICA DE VALÈNCIA

DSIC DEPARTAMENTO DE SISTEMAS INFORMÁTICOS Y COMPUTACIÓN

Escuela Técnica Superior de Ingeniería Informática etsinf

Representación del conocimiento y razonamiento

GRADO EN CIENCIA DE DATOS

Tema 2

Introducción a pyDatalog

Valencia, Septiembre 2026

1. Breve introducción a pyDatalog

pyDatalog es una librería de Python que permite programar en un estilo declarativo utilizando la lógica Datalog, que a su vez es un subconjunto del lenguaje lógico Prolog.

pyDatalog (https://sites.google.com/site/pydatalog/home) es una librería de Python para integrar programación lógica a partir de datos en el lenguaje Python. Con pyDatalog se pueden definir **hechos** y **reglas**, utilizando la sintaxis de lógica de predicados, y luego consultar una base de datos lógica (**base de conocimiento**) como si fueran consultas a una base de datos.

pyDatalog es una herramienta flexible que ofrece muchas funcionalidades (https://sites.google.com/site/pydatalog/Online-datalog-tutorial) como uso de código procedural, diccionarios, conjuntos, consultas sobre datos relacionales, definición de clases con lógica, etc. Sin embargo, para nuestros propósitos nos limitaremos al uso de pyDatalog para definir una base de conocimiento, hacer consultas sobre los componentes de esta base de conocimiento y hacer inferencia lógica a partir de dichos elementos.

1.1 Instalación de pyDatalog

En los laboratorios del DSIC, la librería pyDatalog ya está instalada. Si quieres instalarla en local en tu PC, abre tu terminal o consola de comandos y ejecuta:

```
pip install pyDatalog
```

Para utilizar la librería en tu programa, escribe lo siguiente al comienzo de tu script de Python

```
from pyDatalog import pyDatalog
```

2. Creación de una base de conocimientos

En primer lugar, debemos generar una base de conocimientos donde declaramos todos los hechos (información cierta) que conocemos del entorno.

Supongamos un problema de un sistema con diferentes roles (administrador(a), editor(a), lector(a)) y cada tol tiene ciertos permisos como lectura, escritura y borrado. Además, hay varios usuarios que tienen diferentes roles. Los datos (hechos) disponibles son:

1. El rol de administrador puede leer, escribir y borrar
2. El rol de editor puede leer y escribir
3. El rol de lector solo tiene permiso para leer
4. Marta tiene rol de editora
5. Eloína tiene rol de administradora

6. Eduardo tiene rol de lector

Tenemos que declarar todos los átomos (símbolos) que vamos a utilizar para el problema y esto incluye los símbolos de predicado :

```
from pyDatalog import pyDatalog
pyDatalog.clear()

#símbolos de predicado
pyDatalog.create_terms('permiso,rol')

#hechos: permisos para cada rol (instancias de predicados)
#los hechos se añaden con un símbolo + delante del hecho formando la
#base de conocimiento
+permiso('admin','leer')
+permiso('admin','escribir')
+permiso('admin','borrar')
+permiso('editor','escribir')
+permiso('editor','leer')
+permiso('lector','leer')

#hechos: roles de las persnas
+rol('marta','editor')
+rol('eloina','admin')
+rol('eduardo','lector')
```

Podemos observar que los argumentos de los hechos los hemos declarado como **constantes** (valores fijos) y por eso van entre comillas: `'marta','editor', 'admin',` etc. ya que representan datos fijos y conocidos.

Se pueden usar constantes numéricas sin necesidad de ponerlos entre comillas. Por ejemplo:

```
#añadimos símbolo de predicado edad
pyDatalog.create_terms('permiso,rol,edad')

+edad('marta',33)
```

Conviene aclarar que los símbolos de predicado y las constantes se suelen **escribir en minúscula** en pyDatalog para distinguirlos fácilmente de las variables que se escriben en mayúscula y que veremos en el siguiente apartado. Es una convención heredada del lenguaje de programación lógica Prolog (y por extensión de Datalog).

Las dos instrucciones siguientes muestran por pantalla la veracidad de dos hechos, es decir, comprueban si cada hecho se encuentra en la Base de Conocimientos o no.

```
print(bool(rol('marta','editor')))     #devuelve True
print(bool(rol('marta','lector')))     #devuelve False
```

Para comprobar la veracidad de la información, es decir, para comprobar si un hecho existe en la Base de Conocimiento, podemos emplear el predicado especial **`bool`** de pyDatalog: **`bool(X)`** evalúa el valor lógico de `X` y devuelve `True` o `False`.

3. Ejercicios

Ejercicio 1:

Representa en pyDatalog la siguiente información:

- *Dune* es un libro.
- *1984* es un libro.
- Ana ha leído *Dune*.
- Luis ha leído *1984*.
- *Dune* pertenece al género ciencia ficción.
- *1984* pertenece al género distopía.

Después comprueba si los siguientes hechos pertenecen a la BC:

- Ana ha leído Dune.
- Luis ha leído Dune.
- 1984 es un libro.

Ejercicio 2:

Representa en pyDatalog la siguiente información:

- La línea L1 conecta Benetússer con Valencia.
- La línea L2 conecta Catarroja con Valencia.
- La línea L3 conecta Valencia con Paterna.
- El autobús B10 pertenece a L1.
- El autobús B20 pertenece a L2.
- El autobús B30 pertenece a L3.
- B10 está operativo.
- B30 está operativo.

Realiza tres consultas para comprobar la información en la BC.

4. Creación de reglas para inferencia lógica

En este apartado vamos a ver la creación de una regla para determinar si un usuario tiene permiso para leer, escribir y borrar en función de su rol.

```
from pyDatalog import pyDatalog
pyDatalog.clear()

#símbolos de predicado en minúscula; añadimos
#variables en mayúscula para almacenar Usuario, Permiso y Rol
pyDatalog.create_terms('permiso,rol,tiene_permiso,U,P,R')

#hechos: permisos para cada rol (instancias de predicados)
#los hechos se añaden con un símbolo + delante del hecho formando la
#base de conocimiento
+permiso('admin','leer')
```

```
+permiso('admin','escribir')
+permiso('admin','borrar')
+permiso('editor','escribir')
+permiso('editor','leer')
+permiso('lector','leer')

#hechos: roles de las personas
+rol('marta','editor')
+rol('eloina','admin')
+rol('eduardo','lector')

print(bool(rol('marta','editor')))    #devuelve True
print(bool(rol('marta','lector')))    #devuelve False

#regla para inferir permisos de un usuario según su rol
tiene_permiso(U,P) <= rol(U,R) & permiso(R,P)

#¿tiene permiso marta para escribir?
print(bool(tiene_permiso('marta','escribir')))  #devuelve True
#¿tiene permiso eduardo para borrar?
print(bool(tiene_permiso('eduardo','borrar')))  #devuelve False
```

Analicemos la regla:

```
#regla para inferir permisos de un usuario según su rol
tiene_permiso(U,P) <= rol(U,R) & permiso(R,P)
```

Las condiciones lógicas de la regla se escriben a la derecha del símbolo **<=** y la conclusión a la izquierda. La regla indica lo siguiente: si existe un usuario `U` que tiene el rol `R`, y el rol `R` tiene permiso `P` entonces el usuario `U` tiene permiso para `P`. El operador lógico AND se escribe con el símbolo `&`.

La regla inferirá tantos hechos del tipo `tiene_permiso(U,P)` como combinaciones lógicas de `rol(U,R)` y `permiso(R,P)` sean ciertas. Cada hecho del tipo `tiene_permiso(U,P)` que la regla infiere forma parte del conocimiento derivado de la Base de Conocimientos.

```
#¿tiene permiso marta para escribir?
print(bool(tiene_permiso('marta','escribir')))  #devuelve True
#¿tiene permiso eduardo para borrar?
print(bool(tiene_permiso('eduardo','borrar')))  #devuelve False
```

Como se puede observar en las dos instrucciones print, la aplicación de la regla ha inferido que `'marta'` tiene permiso para `'escribir'`.

Hasta ahora hemos utilizado bool() para comprobar si una afirmación concreta es cierta o falsa. Sin embargo, también podemos realizar **consultas que contengan variables**. En este caso, pyDatalog no devuelve únicamente True o False, sino todos los valores de las variables que hacen cierta la consulta según los hechos y reglas de la base de conocimiento.

Por ejemplo, podemos preguntar qué permisos tiene Marta utilizando la variable `P`:

```
print(tiene_permiso('marta', P))
```

A partir de los hechos y de la regla definidos en la BC, pyDatalog obtiene todos los valores de `P` para los que puede inferirse `tiene_permiso('marta',P)`. Como Marta tiene el rol de editora, se obtienen los permisos 'leer' y 'escribir'.

También podemos dejar las dos posiciones como variables:

```
print(tiene_permiso(U,P))
```

En este caso, pyDatalog obtiene todas las combinaciones de usuario y permiso que pueden inferirse de la base de conocimiento:

U	P
Marta	Leer
Marta	Escribir
Eloina	Leer
Eloina	Escribir
Eloina	Borrar
Eduardo	Leer

Las variables actúan como incógnitas en una consulta. pyDatalog busca los valores que pueden asignarse a dichas variables para que la consulta sea satisfecha por la base de conocimiento.

NOTAS sobre las reglas en pyDatalog:

1. Se pueden escribir tantas condiciones lógicas a la derecha del símbolo `<=` como se quiera
2. Solo se puede escribir una conclusión lógica a la izquierda del símbolo `<=`.
3. Se pueden escribir condiciones lógicas negadas en una regla con el símbolo **~**. Por ejemplo:

```
infiero(X) <= condicion1(X) & ~condicion2(X)
```

4. No se puede usar el operador lógico OR en las condiciones de una regla; el OR se expresa como múltiples definiciones de una misma regla. Por ejemplo:

```
infiero(X) <= condicion1(X)
infiero(X) <= condicion2(X)
```

5. Ejercicios

Ejercicio 1:

Utilizando la BC del ejercicio 1 anterior, añade reglas que permitan representar:

1. Una persona es lectora si ha leído algún libro.
2. Una persona es aficionada a la ciencia ficción si ha leído un libro de ciencia ficción.

3. Una persona es aficionada a un género si ha leído algún libro de ese género.

Realiza las siguientes consultas:

1. Comprueba si Ana es lectora.
2. Comprueba si Luis es lector.
3. Comprueba si Ana es aficionada a la ciencia ficción.
4. Comprueba si Luis es aficionado a la ciencia ficción.
5. Comprueba si Ana es aficionada al género ciencia ficción.
6. Comprueba si Luis es aficionado al género distopía.
7. Obtén todas las personas que son lectoras.
8. Obtén todas las personas aficionadas a algún género e indica a qué género lo son.

Ejercicio 2:

Utilizando la BC del ejercicio 2 anterior, añade reglas que permitan representar:

1. Un autobús conecta dos localidades si pertenece a una línea que conecta esas localidades.
2. Un autobús permite viajar entre dos localidades si conecta esas localidades y además está operativo.
3. Una localidad es accesible desde otra si existe un autobús que permite realizar ese trayecto.

Realiza las siguientes consultas:

1. Comprueba si el autobús B10 conecta Benetússer con Valencia.
2. Comprueba si el autobús B20 conecta Catarroja con Valencia.
3. Comprueba si es posible viajar en el autobús B10 desde Benetússer hasta Valencia.
4. Comprueba si es posible viajar en el autobús B20 desde Catarroja hasta Valencia.
5. Comprueba si Valencia es accesible desde Benetússer.
6. Comprueba si Valencia es accesible desde Catarroja.
7. Comprueba si Paterna es accesible desde Valencia.
8. Obtén todos los trayectos que pueden realizarse actualmente utilizando autobuses operativos.